VERIQO FINAL INTERNAL CHECK ? CLOSURE REPORT

Assessment date: 28 August 2026
Baseline: Round 10 (L99) Level 3 Assessment deliverable
Responds to: "FINAL_INTERNAL_CHECK.docx" ? seven named items (A?G), all explicitly scoped by the reviewer as purely internal engineering requiring no external resources, plus one item
the docx names as hidden ? "hidden P0 terakhir" (the last hidden P0) ? item #12, External Event
Ingestion ? Canonical Case State.
Assessment mode: local engineering qualification and reproducibility review, extended to this round's own explicit instruction: "pastikan semua system diatas dapat berjalan sesuai skema yang diharapkan" (ensure all the above systems run according to the expected scheme).

EXECUTIVE VERDICT

Every one of the seven named items (A?G) and the hidden P0 is closed this round, with real, tested, compiling Go code ? no fabricated external validation, no invented counterparties or experts, matching the discipline every prior round has held to. Two of the eight items (A and G) surfaced real, previously-undetected defects in existing code while being addressed ? not merely gaps in test coverage, but genuine bugs that would have misbehaved in production had they shipped:

- Item A found that casestate.Transition() could reach RESERVED/PAYMENT_AUTHORIZED/ PAYMENT_EXECUTED directly, bypassing the real reserve/payment domain calls those states are supposed to be gated on ? a structural authority/evidence bypass.
- Item G, while proving "duplicate message" handling, found that OpenReserve, AuthorizePayment, and ExecutePayment each performed their real side-effecting domain call (reserve.New, payment.New/Authorize, Payment.Instruct/Settle) *before* reaching the idempotency check ? so a genuine retry (exactly what a caller does after a timeout or network
failure) hit the real domain object a second time and errored outright, rather than returning
the original result as the rest of the system already promised.

Both are fixed, and both fixes are covered by direct regression tests that fail without the fix
and pass with it.

This round does not change the Level 2 / Level 3 posture Round 10 established. Every item in this docx was explicitly internal; none of the 13 genuinely external gates already tracked in
FINAL_MASTER_GAP_MATRIX.json (hsm_kms, live_data, pentest, real historical claim replay, domain expert validation, counterparty network pilot, external registry qualification,
real bank settlement confirmation, and the five operational/infrastructure gates) changed status. They remain honestly BLOCKED_EXTERNAL / READY_FOR_EXTERNAL_QUALIFICATION for the same structural reasons documented since Round 4: they require a real HSM/KMS tenancy, real commercial data contracts, a real independent security vendor, real historical claims, real named domain experts, a real counterparty, real registry credentials, and real physical/ multi-node infrastructure respectively ? none of which a sandboxed engineering session can honestly produce.

ITEMS A?G, ADDRESSED ONE BY ONE

Item	What the docx asked	What this round did
A. State-machine invariant audit	No transition may bypass authority, evidence, audit, payment authorization, or settlement evidence	Closed. Found and fixed a real bypass (see

above). pkg/insurance/casestate: domainCoupledTargets + ErrMustUseDomainCoupledMethod refuse Transition() from reaching the three domain-coupled states directly; ErrSettlementEvidenceRequired refuses leaving PAYMENT_EXECUTED without recorded payment.SettlementEvidence.

B. Cross-domain invariant Payment ? authorized quantum; Reserve == quantum basis; Settlement == payment – documented adjustment; Recovery ? recoverable amount Closed. New pkg/insurance/invariants package, four pure check functions, wired into casestate.OpenReserve/AuthorizePayment/RecordSettlement at acceptance time. Found and fixed a real basis mismatch in the golden case's per-insurer lifecycle (was quantifying against the full co-insurance total while reserving only its own allocated share).

C. Audit completeness Every material event ? a canonical audit event, across all 13 named domains including Recovery and Dispute Closed. pkg/insurance/recovery.Registry gained a real append-only Event history (it had none before ? every mutation was silent); pkg/insurance/auditlink gained MirrorRecoveryHistory and MirrorDisputeMatter, wired into the golden case's unified audit alongside Case/Payment/Reserve/Lifecycle.

D. Reopen semantics CLOSED ? new evidence ? REOPENED ? new version ? new audit chain ? new decision lineage, never rewriting old history Closed. CLOSED ? REOPENED now requires evidence (it previously required none). New Transition.Version, incremented only on a successful reopen, carried through into the canonical ledger; every prior transition's own Version is provably untouched across a reopen.

E. Concurrent actor test Insurer/Broker/Surveyor/P&I sending events simultaneously ? deterministic conflict handling Closed. Two new tests race four distinct actors under -race, 25 trials each, for both the same proposed outcome and different proposed outcomes: exactly one winner every time, every loser refused with a deterministic, typed error.

F. External evidence receipt Every external exchange must carry source, timestamp, issuer, content hash, signature/credential, receipt, verification status Closed. network.ExchangeReceipt enriched with Source, IssuerPartyID, ReceiptReference, CredentialID, and a new ReceiptVerificationStatus vocabulary, plus a Validate() method ? generalizing payment/settlement.go's own SettlementEvidence receipt shape to every external exchange.

G. Failure/retry/idempotency Duplicate, late, out-of-order, timeout, partial exchange, revoked credential, invalid signature, network failure, retry ? never a double transition or double payment Closed. Found and fixed a real double-execution bug (see above). Added ErrOutOfOrderTick (late/out-of-order message) and an opt-in credential.Registry check (ErrActorCredentialNotEffective, revoked credential) alongside the idempotency fix.

Hidden P0 (#12). External Event Ingestion ? Canonical Case State A real, deterministic pipeline: external event ? network adapter ? credential verification ? evidence verification ? canonical event ? case state machine ? SETTLED ? audit ledger ? replayable Closed. New pkg/insurance/ingestion package. IngestExternalSettlement runs all seven named stages in order, fail-closed, on an already-live case, calling only real, already-existing package APIs. Proven by a full-pipeline success test (independently re-verified via a fresh VerifyCanonicalAuthority and Replay call) plus five stage-specific refusal tests, each confirming no partial effect was left behind.

THE TWO REAL DEFECTS FOUND AND FIXED

Both were found not by inspection but by *writing the test the docx asked for and watching it
fail* ? the same discipline that found the Round 10 idempotency and segregation-of-duties bugs.

A. THE DOMAIN-COUPLED BYPASS

Before this round, casestate.Transition(StateReserved, ...) (or PAYMENT_AUTHORIZED/ PAYMENT_EXECUTED) would succeed as long as the caller supplied an authorized role and, where required, an EvidenceID ? with no real reserve.Reserve or payment.PaymentRecord ever created. A case could be observed in RESERVED with no reserve behind it at all. Fixed by refusing these three targets in the exported Transition() outright; only OpenReserve/ AuthorizePayment/ExecutePayment (which perform the real domain call first, under the same lock) may reach them. Replay retains access via the unexported transitionLocked path, since

replaying an *already-recorded* transition is a different question from *creating* a new one.

G. THE RETRY-AFTER-TIMEOUT BUG

OpenReserve, AuthorizePayment, and ExecutePayment each called their real domain method (reserve.New, payment.New+Authorize, Payment.Instruct+Settle) and only *then* called transitionLocked, which is where the idempotency check actually lived. A caller retrying ExecutePayment with the same idempotency key after a timeout ? the single most ordinary retry
scenario in a real payment system ? would have Payment.Instruct refuse a second call (the payment was already PAID, not AUTHORIZED), surfacing a confusing domain error instead of the idempotent success every other transition in this package already provides. Fixed by adding idempotencyPrelude, checked first in all three methods: a genuine retry now returns the original Transition immediately, never touching the domain object again. Covered by TestFailureModesNeverProduceDoubleTransitionOrDoublePayment, which retries ExecutePayment five times with the same key and confirms the underlying Payment is instructed and settled exactly once.

LEVEL 1 / 2 / 3 STATUS (UNCHANGED FRAMEWORK, FROM ROUND 10)

- Level 1 (code implemented): every item A?G and the hidden P0 ? met, with real, compiling, tested Go code.
- Level 2 (cross-domain integrated + replayable): met for every item ? each is wired into the golden case, the canonical audit ledger, or both, and where a state machine is involved (A, D, G, hidden P0), casestate.Replay independently reproduces the same result from recorded history alone.
- Level 3 (real-world / externally qualified): not claimed, not reached, for the same reason Round 10 stated: this docx's own items are explicitly internal, and the 13 genuinely external gates this repository has tracked since Round 4/10 remain open for real-world reasons no engineering session can close.

VERIFICATION PERFORMED THIS ROUND

- go build ./... ? clean.
- go vet ./... ? clean.
- gofmt -l . ? no unformatted files.
- go test ./... ? all packages pass (666+ files, whole repository).
- go test -race ./pkg/insurance/casestate/... ./pkg/insurance/auditlink/... ./pkg/insurance/casepack/... ./pkg/insurance/recovery/... ./pkg/insurance/network/... ./pkg/insurance/invariants/... ./pkg/insurance/ingestion/... ? race-clean, repeated (up to -count=5 on the concurrent-actor tests specifically).
- go test ./pkg/insurance/guardrails/... ? the whole-tree structural guardrail scan (no verdict/liability fields, no opaque confidence floats, no forbidden canonical duplicates, no hard-coded vendor names) ? clean.
- ./scripts/verify.sh ? the project's own CI gate script (build, vet, gofmt, zero-external-dependency invariant, full race+cover test run, 5x race-repeat on consensus-critical packages) ? see test-evidence/r11_verify_output.txt in this deliverable's zip for the full run log.

WHAT CHANGED IN THE DELIVERABLE'S OWN TRACKING ARTIFACTS

- FINAL_MASTER_GAP_MATRIX.json: eight new CLOSED rows (FINAL-CHECK-A through FINAL-CHECK-G, HIDDEN-P0-EXTERNAL-EVENT-INGESTION-PIPELINE), each with real evidence paths, test commands, and acceptance criteria. The 13 pre-existing OPEN/BLOCKED_EXTERNAL rows are unchanged ? they were never in scope for this docx.
- READINESS_MANIFEST.json: regenerated for real via go run ./cmd/veriqo-readiness (never hand-edited) ? see the manifest itself for the current mandatory-gate pass count and level.

HONESTY STATEMENT

No fabricated external validation, no invented counterparties or experts, no simulated live network integration appears anywhere in this round’s work. pkg/insurance/ingestion is explicit, in its own package doc comment, that the "external event" it ingests is a caller-constructed data shape standing in for what a real adapter would hand back ? matching the same interfaces-and-data-shapes discipline pkg/insurance/network has held since Round 8, and payment/settlement.go’s own BankConfirmationAdapter (also interface-only) since Round 10. Every claim of closure in this report is backed by a named test file and a runnable go test command, listed above and in FINAL_MASTER_GAP_MATRIX.json.